

ShardCutter: A Blockchain Sharding Protocol achieving Transaction Workload Balance across State Shards

Huawei Huang ^{ib}, Senior Member, IEEE, Jian Zheng, Xuanye Zhu, Yue Lin, Ting Cai ^{ib}, Member, IEEE, Lu Zhou, Member, IEEE, Zibin Zheng ^{ib}, Fellow, IEEE, Song Guo ^{ib}, Fellow, IEEE

Abstract—Blockchain sharding has been deemed a promising solution that can substantially improve blockchain scalability. However, developers must overcome two major technical challenges to implement a sharded blockchain. The first challenge is the high cross-shard transaction ratio in blockchain shards. This issue significantly degrades the throughput of a sharded blockchain. The second challenge is the imbalanced workloads across blockchain shards. In a blockchain with imbalanced workloads, some busy shards have to handle an overwhelming number of transactions and thus become congested. Facing these two challenges, a dilemma is that it is difficult to guarantee a low cross-shard transaction ratio and maintain the balanced workloads across all shards, simultaneously. We believe that a fine-grained account allocation strategy can address this dilemma. To this end, we formulate the tradeoff between these two metrics as a network-partition problem. We then solve this problem by proposing a sharding protocol, named *ShardCutter*, which includes the following two crucial components: a community-aware account partition algorithm and a fine-tuned account migration mechanism. Finally, experimental results demonstrate that the proposed protocol outperforms other baselines in terms of throughput, makespan, cross-shard transaction ratio, and the workload balance of shards' transaction pool.

Index Terms—Blockchain, Sharding, Account Migration

I. INTRODUCTION

Sharding is viewed as the most promising technique to improve blockchain scalability while not violating the decentralization principle of blockchain systems [3]–[10]. By employing the sharding technique, an original blockchain can be divided into multiple smaller committees, each of which is called a network shard. In a sharded blockchain, each shard verifies and executes transactions (TXs) in parallel, significantly enhancing the throughput.

Sharding techniques are classified into three categories, i.e., network sharding, transaction sharding, and state sharding. Network sharding divides all blockchain nodes into parallel groups in the network layer. Thus, it is the foundation of

This manuscript presents an integrated version of our work by considering two related conference papers, which were presented in SRDS 2022 [1] and accepted by INFOCOM 2024 [2], respectively.

H. Huang, J. Zheng, X. Zhu, Y. Lin, and Z. Zheng are with Sun Yat-Sen University, China. E-mail: {huanghw28, zhizibin}@mail.sysu.edu.cn

T. Cai is with Hubei University of Technology, China. E-mail: caiting@hbut.edu.cn

L. Zhou is with Nanjing University of Aeronautics and Astronautics, China. E-mail: lu.zhou@nuaa.edu.cn

S. Guo (corresponding) is with the Hong Kong University of Science and Technology (HKUST). E-mail: songguo@cse.ust.hk

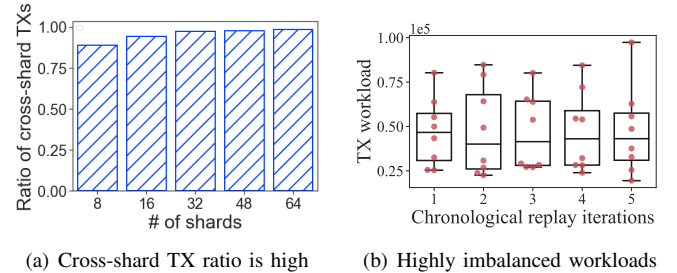


Fig. 1. Results of motivation simulation. Referring to the account assignment of Monoxide [9], we observe a high ratio of cross-shard TXs shown in (a), and imbalanced shard workloads across blockchain shards in (b).

the other two sharding techniques. The difference between transaction sharding and state sharding lies in the storage of the ledger state. Transaction sharding requires every blockchain node to store the whole ledger state, such as the *world state* in Ethereum's blockchain. In contrast, each blockchain node in state sharding only needs to store a subset of the holistic ledger. However, many new technical challenges arise when dividing a large state ledger into multiple smaller ones. The most prominent challenges encompass the atomicity of cross-shard TXs [9] and the related security issues [11] in sharded blockchains. In summary, state sharding is still immature and faces technical challenges when applied to practice.

Motivation. Although several state-of-the-art blockchain sharding solutions [3]–[9] have been proposed, a practical state-reschedule solution, particularly through account migration, is still absent from the literature. Thus, we are curious about what is the challenge behind implementing the account-based state sharding. To disclose valuable information, we conduct the following motivation simulations to emulate the TX assignment in a blockchain sharding system. We collect one million continuous historical TXs from the real-world Ethereum dataset [12]. We then replay them in 5 sequential consensus iterations of consensus. In each iteration, 200,000 (200K) TXs are assigned to the sharded blockchain referring to the assignment policy of Monoxide [9], which is a classic account-based state sharding protocol. To emulate blockchain shards, we divide all consensus nodes into 8 ~ 64 groups. Per the simulation settings depicted above, we have the following two findings.

Finding 1: The ratio of cross-shard TXs is high. As demonstrated in Fig. 1(a), when the number of shards in-

creases, the ratio of cross-shard TX grows and approaches nearly 100%. The overwhelming number of cross-shard TXs could degrade the system throughput and increase the inter-shard communication cost. In Monoxide [9], the authors also mentioned this issue but they did not address it. Monoxide protocol partitions all accounts according to the prefix/suffix of their addresses into different state shards. The problem is that such an account-assignment policy does not consider the correlations between accounts. When accounts with strong correlations are assigned to different shards, it results in a significant number of cross-shard TXs.

Finding 2: The TX workloads across blockchain shards are imbalanced. Fig. 1(b) shows the imbalanced workloads across 8 blockchain shards. Some shards are congested across different replay iterations and some are almost empty. In this paper, we call such congested and almost-empty shards the *hot shards* and *cold shards*, respectively. Those imbalanced TX workloads significantly limit the scalability of a blockchain sharding system. The reasons are twofold. Firstly, the imbalanced workloads force hot shards to store and process more TXs than cold shards. Secondly, the TXs congested in hot shards experience longer makespan than those in cold shards.

Motivated by the two findings mentioned above, we emphasize the following two goals in this paper. i) To avoid a high ratio of cross-shard TXs, we intend to schedule all accounts located in different shards elastically considering their correlations. ii) We also aim to alleviate the workload imbalance across shards. To achieve our goals, we devise an elastic sharding-rescheduling protocol for the blockchain state sharding. Compared with existing sharding solutions, the proposed protocol can not only reallocate accounts across all shards but also improve the scalability of the sharded blockchain while promising its security.

Our study includes the following **contributions**.

- Firstly, to describe the tradeoff between the workload balance across shards and the cross-shard TX ratio, we formulate a network-partition problem.
- To solve this problem, we then propose an elastic load-balancing sharding protocol, named *ShardCutter*, which includes two components: i) a community-aware account-partition algorithm, and ii) a fine-tuned account migration mechanism that enables the sophisticated migration of a specified account across shards. We then rigorously analyzed the property and security of the protocol.
- Finally, we implemented our *ShardCutter* protocol on top of an open-sourced blockchain sharding testbed, Block-Emulator [13]. The evaluation results using the real-world historical Ethereum transactions show that the proposed *ShardCutter* protocol outperforms other baselines regarding throughput, TX makespan, cross-shard TX ratio, and the queue size of shards' TX pools.

The rest of this paper is organized as follows. Section II reviews the related work. Section III presents problem formulation. We then present and analyze the proposed sharding protocol in Sections IV and V, respectively. Section VI shows the evaluation results. Finally, Section VII concludes this paper.

II. RELATED WORK

A. Transaction Models

Blockchain systems currently execute through exploiting two major types of transaction models, i.e., the Unspent Transaction Output (UTXO) model and the account/balance model. Firstly, in a UTXO-based blockchain, users can have multiple UTXOs in their digital wallets. When a transaction spends a certain number of UTXOs, new UTXOs will be generated and can be spent by other future TXs. Such UTXO model is widely adopted by Bitcoin [14] and several sharded blockchains [3]–[5]. Secondly, in an account-based blockchain, each user has their own account. Tokens are deposited in these accounts, as we experience in the banking system. The account/balance model is believed to be more promising than the UTXO model since it supports smart contracts. Furthermore, the TX structure of the account/balance model is more concise because a TX is only associated with one payer account and one payee account.

B. Blockchain Sharding

A blockchain sharding system is operated in *epochs*. Each epoch includes two critical phases: the consensus phase and the reconfiguration phase. In the consensus phase, each shard committee processes TXs locally through intra-shard consensus. In the reconfiguration phase, blockchain nodes may join or leave the system, thus shard committees need to be shuffled under a given policy. Consequently, shard committees are reconfigured to avoid Sybil attacks [19] and Bribery attacks [20]. Elastico [3] is viewed as the first sharding system designed for public blockchain. In Elastico, member committees verify disjoint groups of TXs in parallel. The final committee then includes the verified TX blocks in the main chain. Elastico only achieves *transaction sharding*, which requires every blockchain node to store the holistic state ledger, and thus costs a lot of storage and communication resources. Since then, researchers have paid great attention to *state sharding*, and have proposed several state-of-the-art protocols such as OmniLedger [4], RapidChain [5], and Monoxide [9].

Recently, CoChain [21] and Benzene [22] have been proposed to increase the number of shards while ensuring overall security. Consequently, system throughput increases with the number of shards. MVCom [23] proposes an online stochastic-exploration algorithm to schedule the most valuable committees for the large-scale sharding blockchain. Some emerging studies [6], [24]–[26] are investigating the state sharding of smart contracts, which is a promising research direction.

C. Cross-shard TX Processing in State Sharding

Cross-shard TXs are inevitable in a sharded blockchain [9]. State sharding only requires blockchain nodes in each shard to store a part of the whole state ledger. However, the verification and the execution of cross-shard TXs under state sharding are challenging because the inputs and outputs of a cross-shard TX are located in different shards. Thus, how to process the cross-shard TXs efficiently while guaranteeing their eventual atomicity is a challenge in state sharding.

TABLE I
COMPARISON AMONG DIFFERENT ACCOUNT MIGRATION SCHEMES.

Solutions	Migration Techniques	Implementation in real systems	TXs that need to be locked before account migration
Shard Scheduler [15]	designs an algorithm that can generate account migration recommendations based on account interactions	No	Not Applicable (N/A)
BrokerChain [11]	employs historical TXs to construct a global account graph, which is then subsequently fed into Metis [16] to partition all accounts into appropriate shards	No	N/A
TxAllo [17]	converts the account partition problem to the community detection problem. A deterministic and fast partition scheme is proposed to dynamically determine the accounts' assignment.	No	N/A
Full Lock [18]	facilitates account migration across blockchain shards, and locks the target account's both <i>state</i> and <i>all associated transactions</i> (<i>Payee</i> and <i>Payer</i> transactions), before migration to a new shard.	Yes	All <i>payee</i> and <i>payer</i> TXs
ShardCutter (ours)	facilitates real-time processing of a <i>subset</i> of the affected transactions associated with the target accounts under migration. Fewer TXs are locked in the TX pool than the Full Lock (e.g., [18]).	Yes	<i>Payer</i> TXs only (Benefit: the target account can still receive money during migration)

To deal with such atomicity, some studies [4], [5] built their solutions on top of the UTXO model, while others [9] chose the account-based transaction model. For example, OmniLedger [4] presents a *Byzantine Shard Atomic Commit* protocol to secure the atomicity of cross-shard TXs. However, the bottleneck of OmniLedger is on the client side. RapidChain [5] proposes a *transfer mechanism* to relieve the responsibility of client users. Monoxide [9] exploits the *relay transaction* mechanism, a message-relay TX processing approach. Although relay TXs guarantee the eventual atomicity of cross-shard TXs, those TXs may experience a long makespan.

D. Account Migration

To mitigate the ratio of cross-shard TXs or distribute the workload evenly among shards, some recent studies are exploring *account migration*. In other words, these studies aim to reassign accounts to suitable shards within the framework of account-based blockchain state sharding. The following presents a review of several representative approaches. To help readers have a better comparison among those account migration studies, we present some representative solutions in Table I. Shard Scheduler [15] designs an algorithm that can generate account migration recommendations based on account interactions. Additionally, Shard Scheduler offers an incentive mechanism to encourage the miners to follow the recommendations. BrokerChain [11] employs historical TXs to construct a global account graph. This global graph is subsequently fed into Metis [16] to partition all accounts into appropriate shards. Metis is a classic algorithm for balanced graph edge partitioning. TxAllo [17] converts the account partition problem to the community detection problem. A deterministic and fast partition scheme is proposed to dynamically determine the assignment of accounts. However, the aforementioned approaches primarily concentrate on allocating

accounts to shards without addressing the practical implementation of *account migration*.

Via a thorough review of existing literature, we discovered that only one mechanism [18], referred to as *Full Lock* mechanism, has been proposed for facilitating account migration across blockchain shards. The *Full Lock* mechanism involves the initial locking of the account's state before migration to a new shard. Throughout the migration process, the account is locked, preventing any alterations. TXs attempting to modify the target account are also locked and subsequently moved to the destination shard. Consequently, these TXs endure an extended locking latency, thereby prolonging their makespan.

Compared with the state-of-the-art blockchain sharding solutions [4]–[6], [9], [11], [15], the unique contribution of our proposed sharding protocol lies in that our approach tries to find a tradeoff between the workload balance across all shards and the ratio of cross-shard TXs. Additionally, we consider how account migration impacts the TXs associated with the target accounts being migrated. In contrast to the state-of-the-art *Full Lock* mechanism [18], our proposed *ShardCutter* protocol facilitates the real-time processing of only a subset of these affected transactions. Thus, *ShardCutter* can alleviate the queueing congestion of the TXs associated with those target accounts under migration. This is because only the payer transactions need to be locked for target accounts before migration.

III. SYSTEM MODEL AND PROBLEM STATEMENT

A. The Preliminaries of System Model

The proposed protocol operates within the framework of the account-based transaction model. Following most sharding blockchains [22], [24], [27]–[31], our emphasis lies in state sharding. In our system, each account's state is exclusively managed by one shard at a time. This means that each shard

oversees a distinct subset of all accounts. We assume that blockchain nodes within a shard can access headers from all blocks committed in other shards, allowing each node to verify the validity of an account migrated from another shard. For intra-shard consensus, our system adopts Practical Byzantine Fault Tolerance (PBFT) [32] consensus in this paper. To handle cross-shard TXs, we employ the *relay transaction* mechanism proposed in Monoxide [9].

B. The Impact of Account Partition

As mentioned in Section I, the excessive number of cross-shard TXs and workload imbalance issue are caused partially by the unreasonable partition of accounts. To have a better understanding, we illustrate the effect of account partition using the example shown in Fig. 2. The account network includes 8 accounts and 7 edges. Each edge represents the TX correlation between a pair of accounts. The edge weight denotes the number of TXs occurred between those two associated accounts. For simplicity, we measure the weight of each edge as 1, and suppose the blockchain system has 2 shards. The workload of each shard is calculated as the total number of TXs contributed by the accounts in that shard. We calculate the index of workload imbalance by taking the absolute value of the workload difference between the two shards. As shown in Fig. 2(b), only one cross-shard TX exists in such account partition. However, shard #1 needs to process 7 TXs while shard #2 only needs to process one TX. Thus, the index of workload imbalance is as high as 6. In the next partition illustrated in Fig. 2(c), the TX workloads in both shards are perfectly balanced. However, this account partition strategy results in 7 cross-shard TXs, which are large TX workloads to both shards. The optimal account partition is depicted in Fig. 2(d), where only one cross-shard TX is observed and the workload imbalance index is 0.

From the example illustrated above, it is evident that an effective account-partition solution can not only reduce the number of cross-shard TXs, but also contribute to achieving workload balance across all shards. To better illustrate the impact of account partition, we formulate it as an optimization problem as shown in the following subsection.

C. Problem Formulation of Account Partition

Consider an account network represented by $G(V, E)$, where V is a set of $N \in \mathbb{N}^+$ accounts, denoted by $[N] = \{1, 2, \dots, N\}$; and E is a set of edges connecting these accounts. We use $[K] = \{1, 2, \dots, K\}$ to denote the set of $K \in \mathbb{N}^+$ shards. For each account $i (i \in [N])$, we introduce a binary variable $x_{i,k} (k \in [K])$ to represent the account assignment:

$$x_{i,k} = \begin{cases} 1, & \text{account } i \in [N] \text{ is assigned to shard } k \in [K]; \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

Furthermore, each account i should be assigned to exactly one shard. That is, $\sum_{k=1}^K x_{i,k} = 1, i \in [N]$. We also define $\mathbf{x} = \{x_{i,k}, i \in [N], k \in [K]\}$ to denote the account-partition solution of all account.

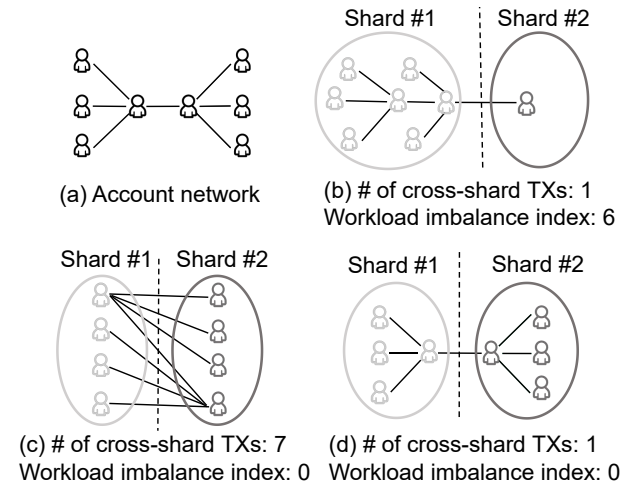


Fig. 2. The effects of different account-partition solutions.

We then use $e_{i,j} \geq 0 (i, j \in [N])$ to denote the edge weight between accounts i and j , indicating the number of TXs that occurred between them. Recall that assigning accounts i and j into two different shards results in an additional number $e_{i,j}$ of relay TXs, thereby increasing TX workloads at the related shards. If accounts i and j are assigned into the same shard, we have $\sum_{k=1}^K x_{i,k} \cdot x_{j,k} = 1$; otherwise, the result is 0. Let $C(\mathbf{x})$ denote the number of cross-shard TXs under an account-partition solution $\mathbf{x}$, we have

$$C(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N [e_{i,j} \cdot (1 - \sum_{k=1}^K x_{i,k} \cdot x_{j,k})]. \quad (2)$$

TX workload of each shard includes both intra-shard and cross-shard TXs. We denote by $L_{k,l}(\mathbf{x})$ the cross-shard TX workload between shards k and l ($k, l \in [K], k \neq l$), which represents the total weight of edges connecting accounts in these two shards. Then $L_{k,l}(\mathbf{x})$ can be calculated as:

$$L_{k,l}(\mathbf{x}) = \sum_{i=1}^N \sum_{j=1}^N e_{i,j} \cdot x_{i,k} \cdot x_{j,l}, \forall k, l \in [K], k \neq l. \quad (3)$$

When it comes to $l = k$, $L_{k,k}(\mathbf{x})$ represents the intra-shard TX workload of shard k , which is expressed as:

$$L_{k,k}(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N e_{i,j} \cdot x_{i,k} \cdot x_{j,k}, \forall k \in [K]. \quad (4)$$

Thus, the total TX workload of shard k , denoted as $T_k(\mathbf{x})$, can be expressed as:

$$T_k(\mathbf{x}) = \sum_{l=1}^K L_{k,l}(\mathbf{x}) = \sum_{l=1, l \neq k}^K \sum_{i=1}^N \sum_{j=1}^N e_{i,j} \cdot x_{i,k} \cdot x_{j,l} + \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N e_{i,j} \cdot x_{i,k} \cdot x_{j,k}, \forall k \in [K]. \quad (5)$$

We further define $\hat{T}(\mathbf{x}) = \frac{1}{K} \sum_{k=1}^K T_k(\mathbf{x})$ as the average workload of the sharding system and $D(\mathbf{x})$ as the system workload imbalance index. A higher value of $D(\mathbf{x})$ indicates

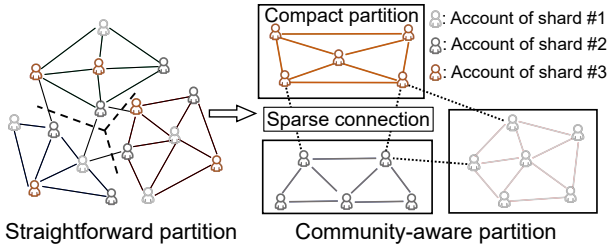


Fig. 3. Straightforward partition vs. community-aware partition.

a more unbalanced system workload. $D(\mathbf{x})$ is computed as:

$$D(\mathbf{x}) = \max_{k \in [K]} |T_k(\mathbf{x}) - \hat{T}(\mathbf{x})|. \quad (6)$$

When given an account network, our goal is to discover an effective account partition solution that minimizes the number of cross-shard TXs and reduces the workload imbalance index to its lowest. We now formulate the account partition problem as the following optimization.

$$\begin{aligned} \min \quad & F(\mathbf{x}) = \alpha \cdot C(\mathbf{x}) + (1 - \alpha) \cdot D(\mathbf{x}) \\ \text{s.t.} \quad & \sum_{k=1}^K x_{i,k} = 1, \quad i \in [N]. \\ \text{Variables:} \quad & x_{i,k} \in \{0, 1\}, \quad \forall i \in [N], \quad k \in [K], \end{aligned} \quad (7)$$

where $\alpha \in [0, 1]$ is a predefined coefficient measuring the weight between the two objective terms $C(\mathbf{x})$ and $D(\mathbf{x})$.

Referring to the conventional graph-partition problems [33]–[35], to find the optimal solution of the above problem is NP-hard [36]. Fortunately, we have observed that the account network exhibits social network characteristics. Thus, we are encouraged to employ a *community detection* mechanism to develop an account-partition algorithm, as elaborated in the following subsection.

D. Community Detection over Blockchain's Account Network

In a complex network, communities are defined as groups of nodes with dense connections, where nodes within each community exhibit strong mutual connections. In contrast, the connections between nodes from different communities are sparse. Communities are indicative of local properties of individual node behavior within a network. Community detection is the process of identifying these communities in a given complex network.

All accounts and transactions in a blockchain construct a social network, with transactions representing connections between accounts. As depicted in Fig. 3, a straightforward partition policy results in a high volume of cross-shard TXs. In contrast, by taking advantage of community properties, we can reduce the number of cross-shard TXs. In a blockchain sharding system, each shard essentially represents a community consisting of multiple accounts. In other words, if we can identify several communities throughout the system and assign each community to a dedicated shard, the number of cross-shard TXs is possibly reduced. Based on the principle mentioned above, we can enhance the performance of the sharding system by devising a community-detection algorithm.

The community detection in social networks has been studied over the past few years. Representative practical community-detection algorithms include label propagation [37], Louvian algorithm [38], and Infomap [39]. However, those state-of-the-art community detection algorithms cannot be directly exploited to solve the proposed account partition problem (7). This is because classical community-detection algorithms do not impose constraints on the size of a community, potentially resulting in very large communities. While an effective community detection algorithm can keep cross-shard TX workloads at a low level, it may also contribute to a high workload imbalance index. Therefore, based on the conventional framework of community-detection methods, we propose a novel community-aware account-partition algorithm, which is then presented in the next section.

IV. SHARDCUTTER: SHARDING PROTOCOL DESIGN

A. The Overview of Protocol ShardCutter

The overview of the proposed sharding protocol is illustrated in Fig. 4. The protocol runs in *epochs*. Each epoch consists of three typical phases: i) *identity setup* phase, ii) *intra-shard consensus* phase, and iii) *account-graph update* phase. The *identity setup* phase must run only once at the beginning of each epoch. After that, a fixed rounds of intra-shard consensus are executed to generate a specific number of transaction blocks. Following the *intra-shard consensus* phase, the *account-graph update* phase must be executed to update the account partition over all W-shards.

Two types of shards exist in the proposed protocol, i.e., the *worker shard* and the *main shard*. They are defined as follows.

- **Worker shard** (abbr., *W-shard*). A worker shard is responsible for processing transactions and generating a certain number of *TX blocks* in each epoch.
- **Main shard** (abbr., *M-shard*). The main shard is responsible for running the proposed account-partition algorithm periodically, and maintaining the latest account-partition decision through generating a *decision block* in each epoch. Thus, M-shard only stores the location of each account, instead of the state of the whole blockchain ledger.

Although the functions of M-shard seem to contradict the decentralized ethos of blockchain systems, the account-partitioning results and the account-migration decision must be verified by all M-shard's consensus nodes. Similar to the *Beacon Chain* in Ethereum 2.0, the *P-shard* designed in [40], and the *transaction allocation service* module proposed in [18], we think the M-shard in our protocol is practical to a blockchain sharding system. In addition, similar to existing sharding systems [3]–[5], [27], our protocol adopts the classic PBFT [32] as the intra-shard consensus protocol. We also adopt the *relay transaction* mechanism [9] to process the cross-shard TXs.

Fundamental steps of the protocol. We present the major steps of the proposed protocol using Fig. 4. Those phases are briefly presented as follows.

- Step ①: W-shards collect newly arrived transactions and generate *TX blocks* in parallel. Then, these TX blocks

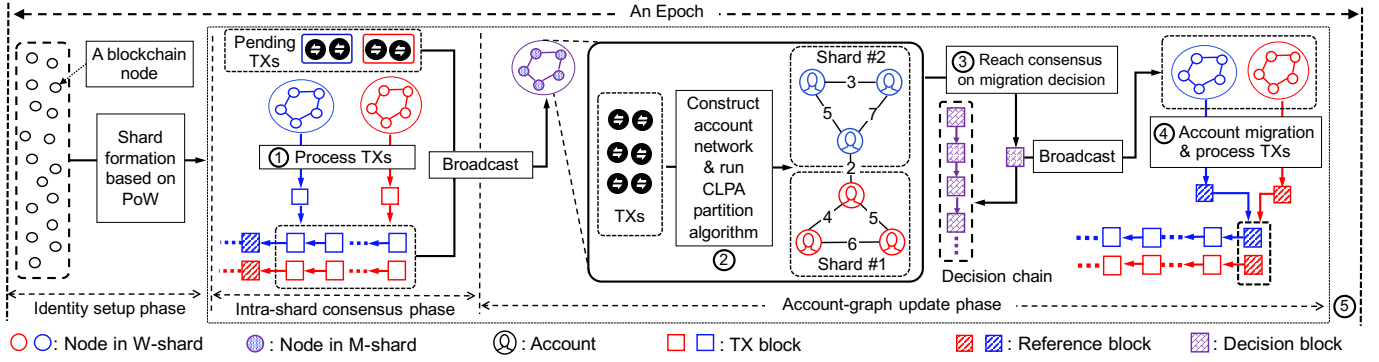


Fig. 4. The overview of the proposed *ShardCutter* sharding protocol.

and the pending TXs are synchronized to M-shard periodically.

- Step ②: M-shard constructs the account network's graph and conducts account partition. M-shard first establishes the account network's graph according to the committed TXs in this period and the pending TXs sent from W-shards. Then, M-shard runs the CLPA algorithm (introduced in Section IV-C) to perform the community-aware partition over the account network's graph.
- Step ③: M-shard reaches consensus on the partition result and generates a *decision block*. Please refer to how an M-shard reaches consensus and updates the *decision chain* in Section V-C of our previous work [1]. Then, M-shard broadcasts the result in terms of *auxiliary TX* (introduced in Section IV-D) to each W-shard.
- Step ④: Each W-shard completes account migration and TX processing at the same time. A W-shard also generates a *reference block* as the *genesis block* for all its shard nodes in the next epoch. Please refer to more details about the design of the *reference block* in Section V-C of our previous work [1].
- Step ⑤: Blockchain system refreshes the formation of both M-shard and W-shards. The sharded blockchain system enters the *identity setup phase* for the next epoch.

B. Identity Setup and Shard Formation

1) *How to Set Shard Formation*: To defend against Sybil attacks in each shard, our protocol entails a key phase, named the *identity setup phase*. We describe its functionality as follows. Before Step ①, all blockchain nodes must solve a hash-based puzzle to establish their identity for *shard formation*. All blockchain nodes that join the same shard essentially join the shard's *committee*. Referring to related studies [4], [5], at the beginning of each epoch, an unpredictable and unbiased random string *Epoch Randomness* (ER) is generated via the verifiable random function (VRF) [41]. Blockchain nodes that intend to join the shard committees need to find a difficulty-specified Proof-of-Work (PoW) solution that can be used to form committee shards. Specifically, each node needs to find a valid *nonce* satisfying the following constraints:

$$\text{Output} = H(\text{ER}||\text{IP}||\text{PK}||\text{Type}||\text{Nonce}) \leq 2^{\gamma-D_{\text{Type}}},$$

where $||$ represents the conjunction operation; and *IP*, *PK* denote the node's IP address and public key, respectively. Parameter γ is the bit-length of the random output of oracle $H(\cdot)$, while D_{Type} is the predefined mining difficulty. The field *Type* denotes which type of shards the node intends to join:

$$\text{Type} = \begin{cases} \text{'M'}, & \text{if the node intends to join M-shard;} \\ \text{'W'}, & \text{if the node intends to join W-shard.} \end{cases}$$

Since M-shard needs fewer nodes than W-shards, the mining difficulty to become a valid member of M-shard D_M is bigger than that of W-shard D_W . We use $\varpi \in \mathbb{N}^+$ and $v \in \mathbb{N}^+$ to denote the number of nodes in an M-shard and a W-shard, respectively. Without loss of generality, we also assume all W-shards have the same number of blockchain nodes. Thus, we set $D_M = D_W + \log_2 \frac{K \cdot v}{\varpi}$, where $K \in \mathbb{N}^+$ is the number of shards. Besides, D_M and D_W can be changed according to real-time mining hashrate to keep the number of nodes stable within a fixed protocol window. The proposed protocol allows nodes to mine for two identities simultaneously to prevent malicious nodes from only establishing identities in the M-shard and W-shards. It implies that a node $\hat{n}$ can fill the field *Type* with both 'M' and 'W'. If the lottery-drawing ticket $\text{Output}_{\hat{n}} \in (2^{\gamma-D_M}, 2^{\gamma-D_W}]$, node $\hat{n}$ joins the W-shard; if $\text{Output}_{\hat{n}} \in [0, 2^{\gamma-D_M}]$, node $\hat{n}$ joins the M-shard. Our design is unbiased for all nodes since they can choose the type of shards they intend to join. The M-shard and W-shards can be formed following a specified sequence, such as the sequence of completing the PoW hash-based puzzle.

2) *The Implications of Failure Probability of an M-shard*: Let $\varphi \in (0, 1)$ denote the proportion of malicious nodes' computational power in the overall system. We then analyze the failure probability of an M-shard when all shards are formed by the PoW mechanism in the *identity setup phase*.

Referring to Theorem 2 of our previous study [40], given the system parameter $\varpi \in \mathbb{N}^+$ as the size of an M-shard (i.e., the number of nodes in an M-shard), blockchain developers can estimate the security level of the blockchain when blockchain shards adopt the PBFT as their internal consensus. That is, the failure probability of an M-shard $\hat{P}$ is written as $\hat{P} =$

$$\sum_{i=\lfloor (\varpi-1)/3 \rfloor + 1}^{\varpi} \binom{\varpi}{i} (\varphi)^i \cdot (1-\varphi)^{\varpi-i}, \quad \varpi \geq 1, \varphi \in (0, 1).$$

Theorem 1 and Theorem 2 of our previous study [40]

imply that the system should **enforce nodes randomly joining different shards**. According to $\hat{P}$, the system can keep an appropriate size of M-shard (i.e., ϖ) to make the probability of the M-shard failure lower than a certain predefined threshold, e.g., $\hat{P} \leq 2^{-\lambda}$, where $\lambda \in \mathbb{N}^+$ is the parameter that describes the security level of the sharding system. For example, when $\lambda = 18$, the probability of the M-shard failure is not greater than $2^{-18} \approx 3.8 \times 10^{-6}$, which means that an M-shard failure is expected to occur once every 2.6×10^5 times of shard-shuffling reconfiguration.

C. Constrained Label Propagation Algorithm (CLPA)

The label propagation algorithm (LPA) is a lightweight community detection algorithm. It has high efficiency in community detection. LPA has been adopted to solve the balanced k -way partition problem [35]. In the original LPA, each vertex is initially assigned a label randomly. Then its label is iteratively updated based on the weight of its neighbors' labels until convergence is reached. Based on the original LPA, we propose the constrained label propagation algorithm (CLPA) to solve the formulated account-partition problem.

The CLPA algorithm takes the account network $G(V, E)$ and the maximum number of iterations t as inputs. In the end, CLPA outputs the account-partition results $V_1, V_2, \dots, V_K$ where $V_i (i \in [K])$ denotes the set of accounts assigned to shard i . Now we elaborate the algorithm. In the initialization phase, $l(i)$ is firstly utilized to denote the label of vertex $i \in [N]$, which is assigned an initial value according to its current shard ID. Then, each vertex traverses its neighbors, and computes a score by invoking the score function $s(i, k)$ to update its new label. We use $\text{NB}(i)$ to represent the neighbor vertexes of vertex i and $\text{K}(i) = \{l(j), j \in \text{NB}(i)\}$ to denote the label set of all its neighbor vertexes. For each iteration, the new label of vertex i is represented by $\arg \max_{k \in \text{K}(i)} s(i, k)$. The score function $s(i, k)$ is defined as follows.

$$s(i, k) = \sum_{j \in \text{NB}(i)} \frac{e_{i,j} \cdot \delta(l(j), k)}{\sum_{h \in \text{NB}(i)} e_{i,h}} (1 - \beta \cdot \frac{W_k}{\min_{h \in [K]} W_h}), \quad (8)$$

$$\forall k \in \text{K}(i), i \in [N],$$

where $\delta(l(j), k)$ is an indicator function. Only when $l(j)=k$ (i.e., the label of vertex j is equal to k), $\delta(l(j), k)=1$; otherwise, $\delta(l(j), k)=0$. In addition, $W_k (k \in \text{K}(i))$ denotes the total weight of edges associated with label k . The edge weight can be computed and updated incrementally according to Eq. (5). Parameter $\beta \in [0, 1]$ is a tunable coefficient that measures the penalty of label k 's edge weight. A lower value of β leads to a lower cross-shard TX workload but a higher workload imbalance index. Through Eq. (8), the proposed algorithm assigns a score of each shard corresponding to a specified account $i \in [N]$. If a community (i.e., shard) is with a large size, the algorithm can assign a low score such that the size of this community is restricted.

The convergence properties of LPA have been discussed in [37]. In contrast, during the updating phase of the proposed CLPA, each vertex (i.e., account) may frequently shift its community between a pair of shards. This behavior would prevent the algorithm from terminating. Therefore, we set a

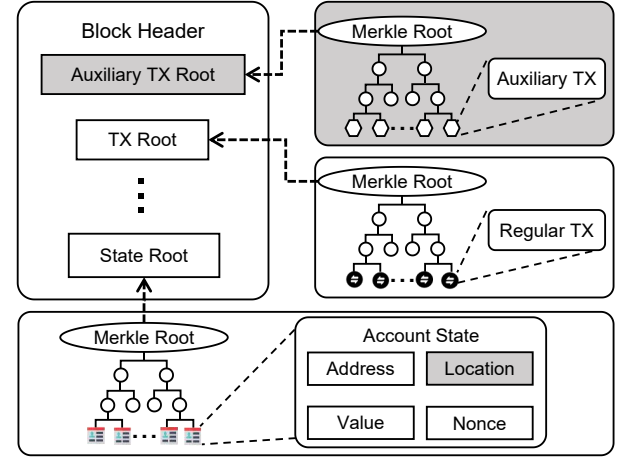


Fig. 5. The data structure of a TX block. The modules highlighted in dark shadows represent the newly added unique components in the proposed protocol.

maximum number of iterations t for CLPA. CLPA algorithm keeps executing iteratively, whether it reaches a convergence or not, within the given t iterations. Thus, the computing complexity of CLPA is $O(N \cdot t)$.

D. The Fine-tuned Account Migration Mechanism

In Step ④ of Fig. 4, each W-shard completes account migration and TX processing at the same time. Since account migration differs from regular transfer transactions, it is necessary to define a new transaction type to assist in completing the account migration, which we call *auxiliary TX*. On the one hand, to complete the account migration as soon as possible, we give auxiliary TX a higher priority than regular TX. On the other hand, we don't expect account migration to hinder the processing of regular TX completely. Therefore, when there are accounts to be migrated, both auxiliary TX and regular TX will be packed in the TX block. Fig. 5 shows the data structure of a TX block in our protocol. In addition to regular TXs, the block body also contains *auxiliary TXs*. Thus, we design a new *Merkle Patricia Trie* (MPT) [12] for indexing auxiliary TXs.

Furthermore, the *Location* component in the account state indicates the shard where the account is located. To determine where an account is located, each shard must maintain the states of all accounts locally. However, if an account is not within the shard, the shard will only store the *Location* field of that account in its state.

Based on the *Full Lock* mechanism [18], when the target account is undergoing migration, the associated TXs must be halted and migrated along with the target account's state, thus extending the makespan of these TXs. In contrast, our proposed protocol permits the execution of *payee TXs* (where the target account is payee) even during the migration of the target account. An illustrative comparison between the state-of-the-art (SOTA) *Full Lock* [18] and the *fine-tuned lock* mechanism of our protocol is presented in Fig. 6.

Fig. 7 shows our protocol's four stages while migrating an account from a source shard to its destination shard.

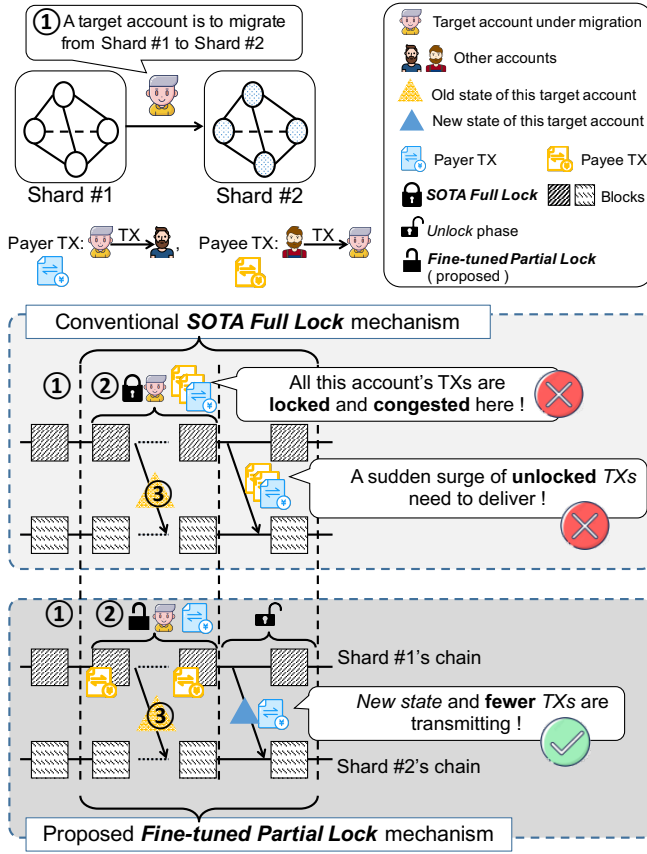


Fig. 6. The comparison between SOTA Full Lock mechanism [18] and the proposed Fine-tuned Partial Lock mechanism working in our ShardCutter protocol. ①: A migration is triggered. ②: Lock phase gets started. ③: Sending old state of this account to Shard #2.

Stage 1. When M-shard makes a decision for account migration, it will send TX_{aux1} to *source shard*. The structure of TX_{aux1} is represented as follows:

$$TX_{aux1} = \{Addr, Source, Dest., \tau\},$$

where field *Addr* denotes the address of the target account, *Source* represents the current shard where the account is located, and field *Dest.* is the shard to which the account is designated. The field $\tau (> 0)$ is the time limit for the migration operation. If the migration is not completed within time τ , it is deemed as a failure. After achieving consensus on TX_{aux1} , *source shard* will not directly map the account to *dest. shard* because this migration may fail due to a timeout. Instead, it updates the *Location* field to '**To Dest. Shard**' in the state of the target account. Afterward, pending *payer TXs* (where the target account is payer) will be locked. Nevertheless, *payee TXs* can continue to be processed in *source shard* until the migration is completed. Then, a TX_{aux2} is sent to *dest. shard*. The structure of TX_{aux2} is defined as:

$$TX_{aux2} = \{TX_{aux1}, State, H, MP_{aux1}, MP_{state}\},$$

where *State* denotes the state of the target account and *H* corresponds to the height of *source shard*'s TX block that contains TX_{aux1} . MP_{aux1} serves as the Merkle Proof to verify

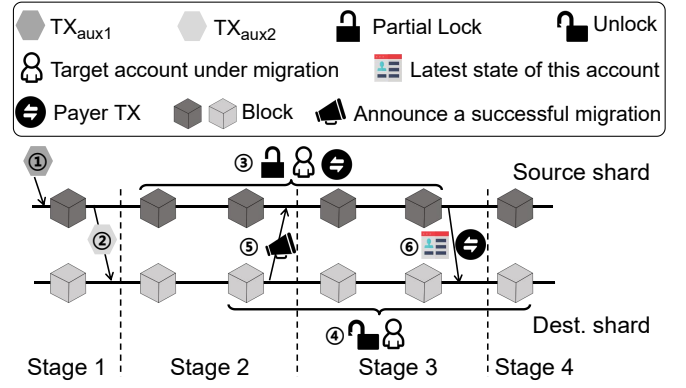


Fig. 7. A successful account migration under our protocol. ①: A migration request TX_{aux1} is sent to Source Shard. ②: Source Shard sends TX_{aux2} that includes the state of the target account to Dest. Shard. ③: Source Shard locks the target account and its associated payer TXs (in which the target account is the payer). ④: Dest. Shard accepts and unlocks the target account. ⑤: Dest. Shard announces this successful migration to all other shards. ⑥: Source Shard sends the latest state of the target account, together with its payer TXs, to the Dest. Shard.

the presence of TX_{aux1} within block *H*, and MP_{state} provides evidence of the account state stored in the same block.

Stage 2. Before packing TX_{aux2} into a block, *dest. shard* will perform a three-step verification process to ensure its legitimacy. Firstly, it checks whether the time limit has been exceeded. Subsequently, it uses MP_{aux1} to confirm whether TX_{aux1} is stored in *source shard*'s block *H*. Finally, *dest. shard* verifies the target account's state by inspecting the *Location* field and then employs MP_{state} to validate its presence in block *H*. If all these validations pass successfully, *dest. shard* will commit the TX_{aux2} and update the *Location* field in the state of the account to '**Dest. Shard**'. *Dest. shard* should record the account's balance *value*₁ at this time, which will be used in *Stage 4*. Afterward, a TX_{ann} will be sent to all shards notifying that the target account has been relocated to *dest. shard*. The structure of TX_{ann} is illustrated as follows:

$$TX_{ann} = \{TX_{aux2}, State, H, MP_{aux2}, MP_{state}\},$$

where *State* represents the target account's state stored in *dest. shard* and *H* refers to the height of the block that stores TX_{aux2} . Additionally, MP_{aux2} and MP_{state} serve as corresponding Merkle Proofs to prove the existence of TX_{aux2} and *State* in block *H*, respectively. *Dest. shard* then begins to process TXs related to the target account.

Note that, if the verification fails, *dest. shard* will notify *source shard*, which will subsequently update the *Location* field in the target account's state to '**Source Shard**' and unlock related pending TXs. Finally, this indicates that the migration for the account is unsuccessful.

Stage 3. Upon receiving the announcement from *dest. shard*, *source shard* will run a consensus on the announcement. Subsequently, it will update the *Location* field of the account's state to '**Dest. Shard**'. This signifies that *source shard* will no longer handle any TXs related to the target account. Finally, *source shard* sends all locked *payer TXs* and TX_{ns} , which includes the latest account state, to *dest. shard*.

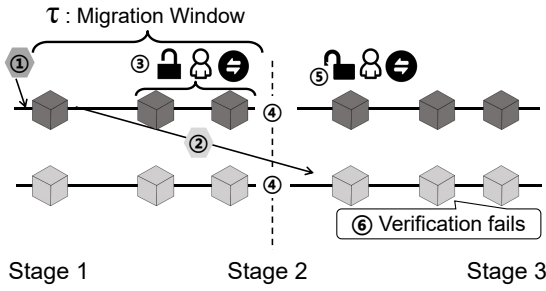


Fig. 8. Demonstration of **type-1 failure**, i.e., migration window τ expires. ④: Migration window expires before TX_{aux2} arrives at Dest. Shard. ⑤: Source Shard unlocks the target account and the associated payer TXs. ⑥: Dest. Shard fails to verify the target account because the migration window τ expires.

The structure of TX_{ns} is designed as follows:

$$TX_{ns} = \{TX_{ann}, NewState, H, MP_{ann}, MP_{newstate}\},$$

where $NewState$ is the latest state of the target account stored in *source shard*, while H signifies to the height of the block containing TX_{ann} . Furthermore, MP_{ann} and $MP_{newstate}$ are corresponding Merkle Proofs for TX_{ann} and $NewState$ in block H , respectively.

Stage 4. Once achieving consensus on TX_{ns} sent from *source shard*, the *dest. shard* firstly records the balance $value_2$. Then, it increments the balance in the account's state stored in it by $|value_2 - value_1|$. Then, the pending TXs sent from *source shard* will be put into the *mempool* of *dest. shard*.

E. Handling Two Types of Migration Failures

During the account migration, our protocol could be threatened by two types of failures.

- **Type-1 failure:** The migration window τ expires in both *source* and *destination* shards.
- **Type-2 failure:** The *destination shard* successfully verifies the account migration, but the *source shard* does not perceive it because the migration window τ expires.

Fig. 8 illustrates how the migration of a **type-1** failure occurs. TX_{aux2} may arrive at *dest. shard* at a late time when the migration window τ has already expired. Thus, *dest. shard* will not pack TX_{aux2} into a block when failing to verify the target account. Moreover, *source shard* will not receive the success announcement within the migration window τ , and the migration is regarded as failed. *Source shard* will unlock the target account by changing the `Location` field in the account's state from '*To Dest. Shard*' back to '*Source Shard*'. Also, the pending TXs in the *locking pool* will be put back into the *mempool*.

Fig. 9 demonstrates a **type-2** failure scenario. In this instance, TX_{aux2} arrives at *dest. shard* in time, enabling successful validation and subsequent broadcasting of the corresponding TX_{ann} . However, due to the propagation delay, TX_{ann} may reach *source shard* when the migration window τ has expired. Consequently, *source shard* prematurely unlocks the target account, leading to an incorrect system status where the target account is duplicated in both shards. This result

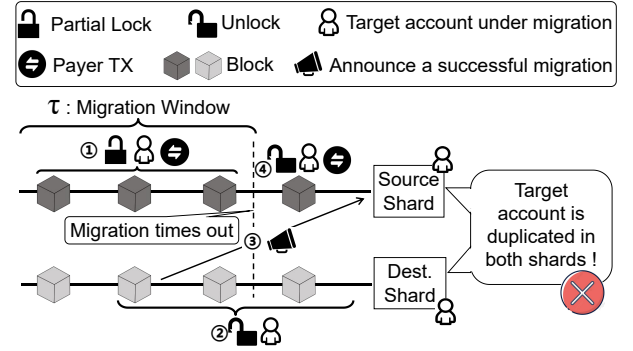


Fig. 9. Demonstration of **type-2 failure**. Consequently, the target account is duplicated in both the source and destination shards.

violates the fundamental principle that an account should be managed by only one shard at any specific time.

To address the **type-2** failure, a restoration strategy is proposed and described as follows. When the migration window τ expires, *source shard* should not unlock the target account directly. Instead, the *source shard* should initiate a new inquiry from the *dest. shard* to confirm the success or failure status of the target account's migration. This process continues periodically until the correct result is perceived by the *source shard*.

V. PROPERTY AND SECURITY OF OUR PROTOCOL

In this section, we mainly present the theoretical analysis of account migration, including *account migration liveness* and *eventual consistency*. Then, we are motivated to analyze the security of M-shards, W-shards, and the account-partition decision in a more general approach compared to our previous works [1], [40]. Please refer to the detailed theoretical analysis from our previous works, i.e., Section VI (Security Analysis of Proposed Protocol) in [1] and Section VI (Security Analysis of Shards) in [40]. Due to space limitations, we present some unique property analysis of the proposed protocol here.

A. Properties of ShardCutter Protocol

Theorem 1. (Migration Liveness) Suppose that the source and the destination shards are not corrupted by malicious nodes, account migration can either be successful or unsuccessful, with no intermediate state.

Proof. Assuming *dest. shard* remains undominated by malicious nodes and the verification of TX_{aux2} succeeds, it will broadcast a notification indicating a successful migration. On the contrary, if the verification fails, it will send a failure message to *source shard*. Upon receiving this failure message, *source shard*, provided it is not corrupted by malicious nodes, will roll back this migration. In summary, account migration can either be successful or unsuccessful. Thus, Theorem 1 concludes. $\square$

Theorem 2. (Eventual Consistency of Migration) Suppose that the M-shard, the source and the destination shards remain undominated by malicious nodes, all shards will eventually share the same and correct value in the `Location` for the target account, regardless of the success of the migration.

Proof. Assuming that *M-shard* is not corrupted by malicious nodes, it will not send two TX_{aux1} migrating the same account to different shards. And if *dest. shard* fails to commit TX_{aux2} , it will notice *source shard*. Even if a malicious entity in *dest. shard* sends TX_{ann} to other shards, they will disregard it due to the absence of a valid Merkle Proof confirming the commitment of TX_{aux2} . Upon receiving the failure message, *source shard* will roll back the migration, only if it is not corrupted by malicious nodes. Consequently, all shards will continue to perceive the account as managed by *source shard*. On the contrary, if the TX_{aux2} is committed by *dest. shard*, and as long as it is not corrupted by malicious nodes, honest nodes will send valid announcements to other shards. Eventually, all shards will redirect the account to *dest. shard*. In summary, all shards share the same and correct value in the `Location` for the target account. $\square$

Remark 1. Theorems 1 and 2 imply that to guarantee the security of account migration, *M-shard*, the *source shard*, and *dest. shard* should remain undominated by malicious nodes. To achieve this, it is necessary to ensure that the percentage of malicious nodes within these shards does not exceed $1/3$, as our protocol utilizes PBFT for intra-shard consensus.

Next, we calculate the probability that *M-shard*, *source shard*, and *dest. shard* are all secure.

Theorem 3. Given a total number of blockchain nodes represented by $O \in \mathbb{N}^+$, with $M \in \mathbb{N}^+$ denoting the number of malicious nodes, each shard consisting of $n \in \mathbb{N}^+$ ($n < O$) nodes, each shard capable of tolerating up to $f \in \mathbb{N}^+$ ($f < n$) malicious nodes, and ω, ς and $\sigma \in \mathbb{N}^+$ ($\omega, \varsigma, \sigma < n$) representing the number of malicious nodes in *M-shard*, *source shard* and *dest. shard* respectively, the probability that *M-shard*, *source* and *dest. shards* remain undominated by malicious nodes is:

$$P = \sum_{\omega=0}^{\min\{f,M\}} \sum_{\varsigma=0}^{\min\{f,M-\omega\}} \sum_{\sigma=0}^{\min\{f,M-\omega-\varsigma\}} \frac{\binom{n}{\omega} \binom{n}{\varsigma} \binom{n}{\sigma} \binom{O-3n}{M-\omega-\varsigma-\sigma}}{\binom{O}{M}}. \quad (9)$$

Proof. Firstly, the probability that the number $m \in \mathbb{N}^+$ ($0 \leq m \leq M$) of malicious nodes in a shard is no larger than f can be calculated as:

$$P(m \leq f) = \sum_{m=0}^{\min\{f,M\}} \frac{\binom{M}{m} \binom{O-M}{n-m}}{\binom{O}{n}}. \quad (10)$$

Then, the probability that there are ω malicious nodes in *M-shard*, ς malicious nodes in *source shard*, and σ malicious nodes in *dest. shard* can be represented as:

$$\begin{aligned} P(\omega, \varsigma, \sigma) &= \frac{\binom{M}{\omega} \binom{O-M}{n-\omega}}{\binom{O}{n}} \times \frac{\binom{M-\omega}{\varsigma} \binom{O-n-(M-\omega)}{n-\varsigma}}{\binom{O-n}{n}} \\ &\times \frac{\binom{M-\omega-\varsigma}{\sigma} \binom{O-2n-(M-\omega-\varsigma)}{n-\sigma}}{\binom{O-2n}{n}} \\ &= \frac{\binom{n}{\omega} \binom{n}{\varsigma} \binom{n}{\sigma} \binom{O-3n}{M-\omega-\varsigma-\sigma}}{\binom{O}{M}}. \end{aligned} \quad (11)$$

Thus, the probability that *M-shard*, *source shard*, and *dest. shard* are all secure, can be calculated as:

$$P(\omega \leq f, \varsigma \leq f, \sigma \leq f) = \sum_{\omega=0}^{\min\{f,M\}} \sum_{\varsigma=0}^{\min\{f,M-\omega\}} \sum_{\sigma=0}^{\min\{f,M-\omega-\varsigma\}} P(\omega, \varsigma, \sigma). \quad (12)$$

Finally, taking Eq. (11) in Eq. (12), Theorem 3 concludes. $\square$

Remark 2. It can be seen from Eq. (9) that, when the total number of nodes O and the number of malicious nodes M are held constant, a judicious selection of the shard size, denoted as n , can significantly enhance the security of *M-shard*, *source shard* and *dest. shard* with a high level of certainty. For instance, using PBFT as the intra-shard consensus (i.e., $f=1/3$), when $O=1000$, $M=150$, and $n=100$, the probability P is over 99.999%.

B. Threat Model of Replay Attack and Defense Solution

In the proposed protocol, we employ the *relay transaction* mechanism [9] to manage cross-shard TXs (i.e., the TXs where the payer and payee are located in different shards). In the original *relay transaction* mechanism, when the payer's shard processes a cross-shard TX (referred to as *CTX*), nodes in the shard perform the deduction operation and subsequently generate a relay TX. The relay TX is then transmitted to the payee's shard, along with the Merkle Proof confirming the commitment of *CTX* in the payer's shard. Upon the successful commitment of the relay TX in the payee's shard, the deposit operation is triggered. However, within a system facilitating account migration, a potential vulnerability arises if the payee's account is undergoing migration. In such a scenario, a malicious adversary in the payer's shard can launch an attack by dispatching the relay TX to both the *source* and *dest. shards*. This malicious behavior is called **replay attack**, which is demonstrated in Fig. 10. This type of *replay attack* inflates the blockchain system because the malicious relay TXs generate an extra volume of tokens.

We then give a formal threat model of the *replay attack* and its defense strategy.

1) *System and Adversary Assumptions:* We consider a sharded blockchain system with the following properties:

- The system consists of $S \in \mathbb{N}^+$ shards, each running PBFT consensus with $n \in \mathbb{N}^+$ nodes per shard.
- At most $f < n/3$ Byzantine nodes per shard (standard PBFT assumption).
- Network is partially synchronous (messages delayed by at most $\Delta > 0$ time).
- Account migration follows the protocol described in Section IV-D.

2) *Adversarial Capabilities:* The adversary $\mathcal{A}$ can: i) control up to f Byzantine nodes in any shard, ii) eavesdrop on all network communications, iii) delay or reorder messages between honest nodes, iv) inject new messages into the network, and v) initiate account migration requests for any account.

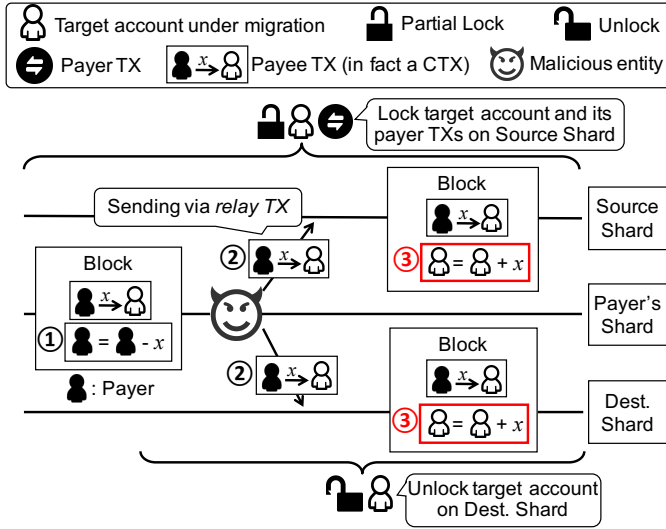


Fig. 10. Demonstration of *Replay Attack*. ①: A *payee* TX (actually a cross-shard TX) is packed in a block of Payer's Shard. ②: A malicious entity in Payer's Shard sends two relay TXs (TX_{relay}) to both Source Shard and Dest. Shard. ③: Both shards deposit a volume x of tokens to the target account.

3) *Replay Attack Scenario*: As illustrated in Fig. 11, during account migration from shard S_1 (i.e., *source shard*) to S_2 (i.e., *dest. shard*), $\mathcal{A}$ can attempt a replay attack when: i) a cross-shard transaction CTX involves account A (payee account) being migrated; ii) TX_{ann} has been received by the payer's shard but not yet by S_1 ; and iii) The time window satisfies: $t_{commit}(CTX) < t_{receive}(TX_{ann}) + \Delta$.

4) *Attack Execution*: $\mathcal{A}$ performs the following steps: i) intercepts relay transaction TX_{relay} from payer's shard; ii) replicates TX_{relay} and sends to both S_1 and S_2 ; iii) causes double execution of the deposit operation if:

- S_1 still shows $A.Location = S_1$;
- S_2 shows $A.Location = S_2$;
- Neither shard can detect the concurrent execution.

5) *Security Requirements*: The protocol must guarantee the following requirements.

- *No Double-Spending*: $\forall CTX$, the deposit operation executes exactly once.
- *Migration Atomicity*: Account A appears in exactly one shard at any time.
- *Liveness*: Valid transactions eventually execute despite migrations.

6) *Defense Invariants*: Our solution maintains the following invariants.

- *Location Consistency*: After TX_{ann} , all shards agree on A 's location.
- *State-Proof Binding*: TX_{relay} includes cryptographic proof of A 's current location.
- *Verification Completeness*: Shards verify both transaction validity and the account's location.

7) *Defending against Replay Attacks*: To defend against the *replay attack* described above, a careful restructuring of the *relay transaction* format is essential. We thus propose the

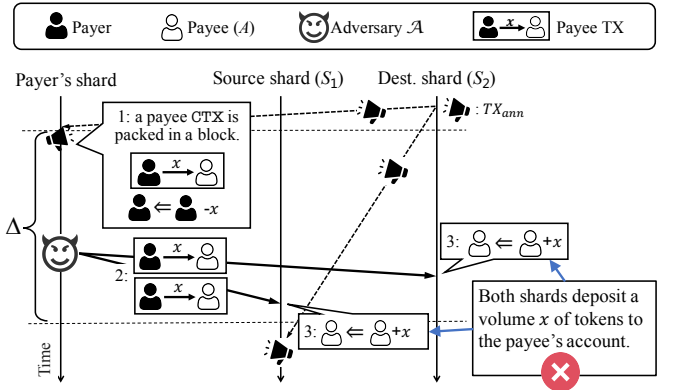


Fig. 11. Threat model of *Replay Attack*. Step 1: A *payee* TX (i.e., CTX) is packed in a block of Payer's Shard. Step 2: Adversary $\mathcal{A}$ in Payer's Shard sends two relay transactions (TX_{relay}) to both Source Shard S_1 and Dest. Shard S_2 . Step 3: Both shards deposit a volume x of tokens to the target account (i.e., the payee account A).

following revised data structure:

$$TX_{relay} = \{CTX, State_{rec}, H, MP_{ctx}, MP_{rec}\}.$$

Within this structure, $State_{rec}$ and MP_{rec} are new attributes differing from the original data structure of TX_{relay} . Specifically, $State_{rec}$ represents the payee's state stored in the payer's shard (only the *Location* field has been filled), H denotes the height of the block committing CTX, MP_{ctx} and MP_{rec} represent the Merkle Proofs that prove CTX and the payee account state in that block, respectively.

Note that, nodes within the payer's shard update the *Location* of the payee to '*Dest. Shard*' upon receiving an announcement from *dest. shard*. Consequently, if *source shard* receives the relay transaction sent from the payer's shard, the relay transaction will be aborted due to a mismatch in the *Location* field, which should be '*Source Shard*'.

In addition, if a valid relay TX arrives at *source shard* after a potential delay, it is possible that *source shard* has successfully migrated the payee to *dest. shard*. In such a case, nodes in *source shard* will transmit the relay transaction to the *dest. shard* along with a Merkle Proof. This proof serves to demonstrate that the *Location* field of the payee's state in *source shard* has been updated to '*Dest. Shard*'.

VI. PERFORMANCE EVALUATION

We implemented the prototype of *ShardCutter* through over 5000 lines of codes using Golang on top of an open-sourced blockchain testbed named *BlockEmulator* [13], which is a blockchain emulator that features blockchain sharding functionality.

A. Settings

Datasets. We adopt the first 1 million TXs from Ethereum's historical transactions [12] as the dataset for our experiments.

Transaction processing. TX blocks generated by W-shards can contain a maximum of 2000 TXs (including regular and auxiliary TXs). The number budget for auxiliary TXs in a block is fixed at 1000. Our experiments exploit a client to

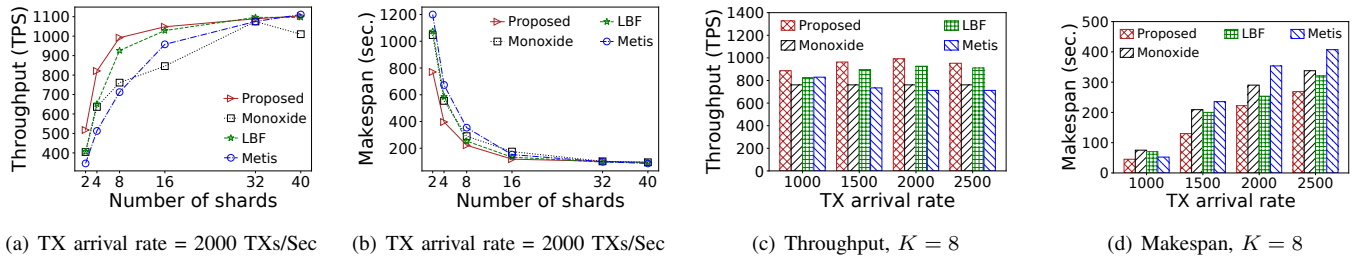


Fig. 12. Throughput and makespan performance while varying the TX arrival rate and the # of shards.

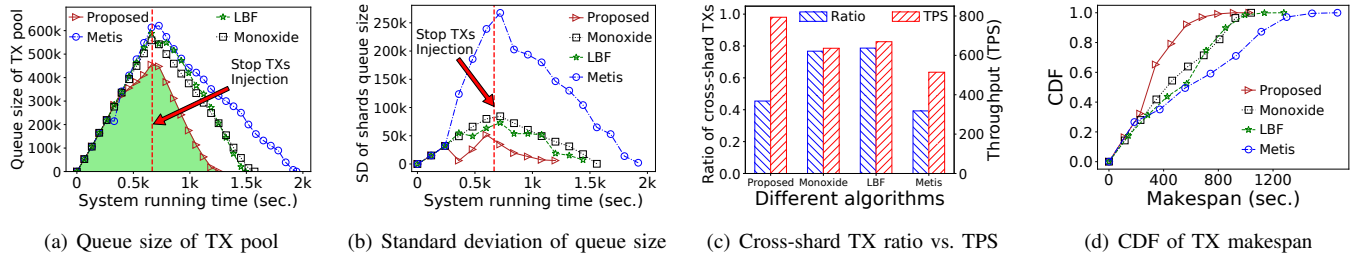


Fig. 13. Multiple metrics while fixing $K = 4$ and TX arrival rate = 1500 TXs/Sec.

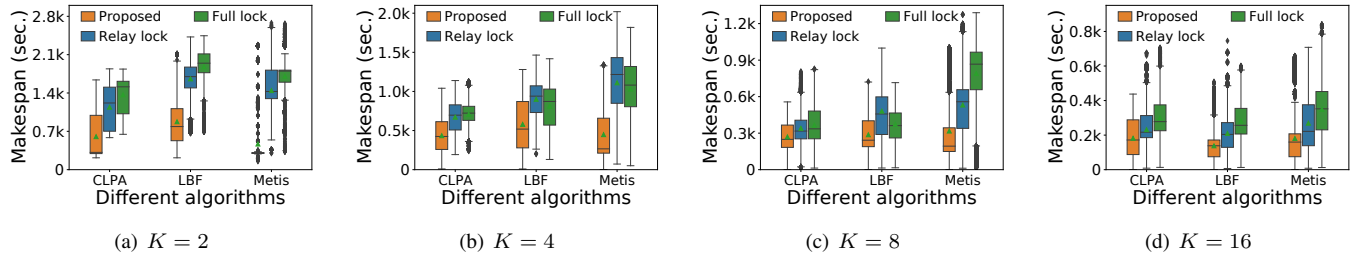


Fig. 14. Makespan of *payee* TXs while fixing TX arrival rate = 2000 TXs/Sec and varying the # of shards.

consistently inject sequential TXs into the blockchain system at a predefined rate. Each shard node arranges the incoming TXs following their arrival sequence.

Baselines for partitioning account network. We compare *ShardCutter* with the following three state-of-the-art workload balance methods for blockchain shards. *Monoxide* [9] is a classic sharding protocol for handling cross-shard TXs. *Metis* [34] is a well-known K -way partition algorithm that can partition the network into K parts with almost equal vertex sizes while minimizing the weight of cutting edges. *LBF* (Load-Balance First) [36] emphasizes the goal of workload balance, but ignores handling cross-shard TXs.

Baselines for migrating accounts. When handling the account migration, we choose three baselines. In addition to the *Full Lock* mechanism proposed in *LB-Chain* [18], we also implement another lock mechanism called *Relay Lock*, inspired by the relay transaction mechanism proposed in *Monoxide* [9]. In the *Relay Lock* mechanism, when the payee account is under migration, *payee TXs* can be locked after the deduction operation is executed on the payer. The proposed fine-tuned account migration mechanism is named *Partial Lock* in experimental results.

Deploying our prototype and BlockEmulator in Tencent Cloud. We rent 68 virtual machines (VMs) from Tencent Cloud. We deployed a blockchain node in each VM node.

Each of these VMs is equipped with 16 CPU cores (Intel Xeon Cascade Lake 8255C) and 32 GB of memory. The network bandwidth is configured to 20 Mbps. The W-shard count is set to 2 ~ 16, and each shard (M-shard or W-shard) consists of 4 blockchain nodes.

Other parameter settings. We set the block interval to 6 seconds. Every 5 minutes, a round of account migration is invoked by M-shard. By default, β and t (parameters for CLPA defined in Section IV-C) are set as 0.5 and 10, respectively.

B. Throughput (measured in TPS)

We first evaluate throughput measured in TXs per second (TPS), while varying the number of shards and TX arrival rates. As shown in Fig. 12(a), we fix the TX arrival rate as 2000 TXs/Sec and vary the number of shards from 2 to 40. We observe that the system throughput increases with the number of shards. The proposed *ShardCutter* always exhibits the highest throughput. When the number of shards is 4, the TPS of *ShardCutter* is around 810, while *Monoxide* needs 16 shards to reach the same TPS. When the number of shards is 8, the TPS of *ShardCutter* is around 1000, and *LBF* achieves around 920, while *Monoxide* and *Metis* only achieve 760 and 710, respectively. Taking the advantages of *ShardCutter*, the throughput is 22% higher than that of *Monoxide* on average when the number of shards is less than 32. In Fig. 12(c), we

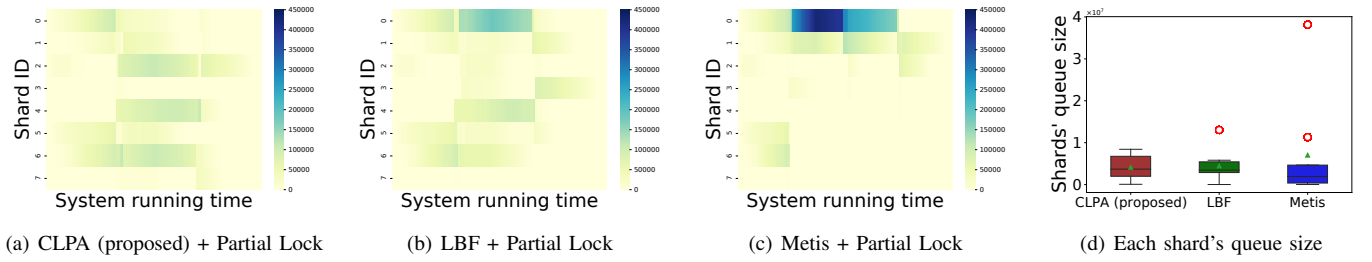


Fig. 15. Queue size of each shard under the proposed Partial Lock mechanism while fixing $K = 8$ and TX arrival rate = 2000 TXs/Sec. Figure (d) represents the distribution of the queue size in each shard across the system running time, where the queue size is an accumulation of pending transactions measured every 6 seconds. When Metis is exploited as the account-partition algorithm, TXs are always allocated into only one shard, thus resulting in an imbalance across shards. In contrast, our proposed (CLPA + Partial Lock) mechanism yields a balanced workload distribution.

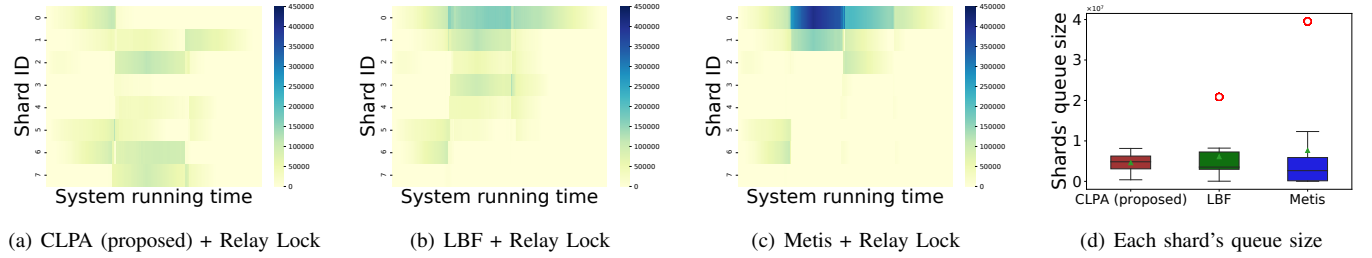


Fig. 16. Queue size of each shard under Relay Lock mechanism while fixing $K = 8$ and TX arrival rate = 2000 TXs/Sec.

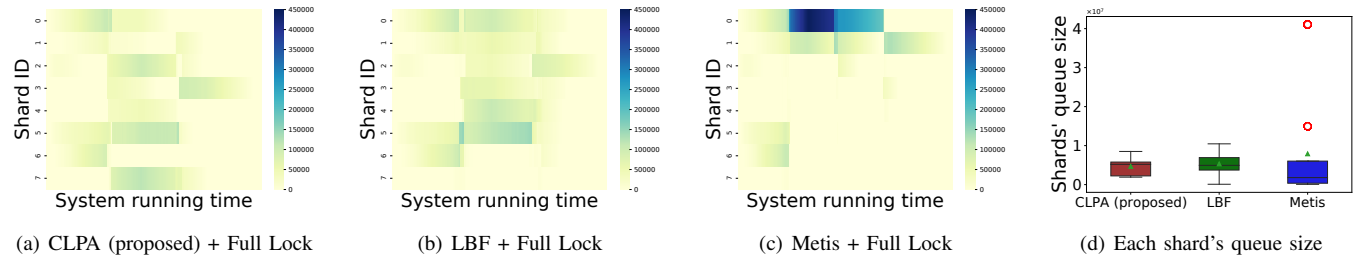


Fig. 17. Queue size of each shard under SOTA Lock mechanism while fixing $K = 8$ and TX arrival rate = 2000 TXs/Sec.

set K as 8 and vary the TX arrival rate from 1000 TXs/Sec to 2500 TXs/Sec. With the increase of TX arrival rate, the TPS of Monoxide and Metis does not increase, indicating that the TX arrival rate has exceeded TX processing speed. When the TX arrival rate exceeds 1000, the TPS of *ShardCutter* and LBF both increase, indicating that the TX arrival rate of 1000 has not reached the limit of how fast the system can process TXs.

C. TX Makespan

We then measure the average TX makespan of different methods under the same parameters used for throughput evaluation. As demonstrated in Fig. 12(b) and Fig. 12(d), under various configurations, the proposed sharding protocol always maintains a low makespan. In Fig. 12(b), we observe that makespan shows a decreasing trend following the increasing number of shards. In Fig. 12(d), when the TX arrival rate increases from 1000 TXs/Sec to 2500 TXs/Sec, the proposed *ShardCutter* protocol maintains almost 30% lower makespan compared with the other three baselines.

D. TX-Pool Size, Cross-shard TX Ratio and Workload Balance

In the following experiment, we fix the number of shards to 4, and set the TX arrival rates to 1500 TXs/Sec. As illustrated in Fig. 13(a), the queue size of the TX pool under all methods keeps growing when new transactions are being injected into the blockchain system. We also observe that *ShardCutter* maintains a more stable and shorter queue than Metis, Monoxide, and LBF.

We are also curious about the performance of throughput, makespan, cross-shard TX ratio, and workload balance across all shards. Thus, we dive deep into more insights. To evaluate the workload-balance metric of different methods, we study the Standard Deviation (SD) of the shard's queue size across time. Fig. 13(b) shows that the proposed *ShardCutter* maintains the lowest SD of local queue size compared with other baselines. This result proves that *ShardCutter* has the best workload balance performance among all methods. In contrast, Metis leads to large fluctuating SDs of the shard's queue size. This observation indicates that the Metis's workload imbalance index is extensive. Metis only aims to reduce the number of cross-shard TXs without considering the workload balance.

From the results shown in Fig. 13(c), we see that the proposed *ShardCutter* and Metis can produce fewer cross-shard TXs than Monoxide and LBF. Though Metis results in the fewest cross-shard TXs, its other performance metrics are almost the worst as shown in Fig. 12 and Fig. 13. The reason is that Metis does not consider the workload balance among shards. Although LBF maintains a low SD of local queue size, it generates the highest cross-shard TX ratio. Thus, the workload balance and the cross-shard TX ratio are tradeoffs in the context of blockchain state sharding. In contrast, the proposed *ShardCutter* protocol can maintain a low ratio of cross-shard TXs and has the highest throughput among all methods.

Recall that hot shards degrade the scalability of a blockchain sharding system, because the congested hot shards cause large TX makespan. From Fig. 13(c), Fig. 13(b) and Fig. 12(d), we find that workload imbalance not only degrades the system throughput, but also might notably increase the TX makespan. To prove our speculation, we studied the cumulative distribution function (CDF) of the TX makespan using different methods. From Fig. 13(d), we observe that up to 80% of TXs are confirmed within 410 seconds under the proposed *ShardCutter* protocol. For other baselines, within the same makespan window, only 52%, 42%, and 40% of TXs are confirmed under Monoxide, LBF, and Metis, respectively. Furthermore, *ShardCutter* protocol spends around 1028 seconds to confirm all TXs while Monoxide, LBF, and Metis consume 1030, 1280, and 1678 seconds, respectively.

E. The Effect of Account Migration on Payee TXs

In the following experiments, we evaluate the impact of account migration on the *payee TXs* under different combinations of partition algorithms and lock mechanisms. We set the TX arrival rates to 2000 TXs/Sec and varied the number of shards from 2 to 16. We name our proposed account migration mechanism as *Partial Lock*.

Fig. 14 shows the makespan of payee TXs under varying settings. The makespan of *payee TXs* decreases as the number of shards increases. Nonetheless, no matter which partition algorithm is used, the makespan of *payee TXs* in *Partial Lock* mechanism remains the shortest. The makespan in other mechanisms contains not only the period when the *payee TXs* are locked but also the queueing time in the *dest. shard*. Furthermore, we can observe that when using Metis as the partition algorithm, the advantage of the proposed *Partial Lock* mechanism is more obvious. This is because TXs that are not locked are processed on time, while locked TXs will later enter the crowded pool of only a few shards due to the imbalanced partition yielded by Metis.

To verify the ideas mentioned above, we investigate the changes in the queue size of each shard over the system's running time. As shown in Fig. 15, Fig. 16, and Fig. 17, the queue size of each shard is more balanced when using CLPA and LBF than in the case under Metis. When Metis is adopted, TXs are almost congested in Shard #0. This results in payee TXs being suspended in this crowded shard. Figs. 15(d), 16(d), and 17(d) illustrate that outliers are shown in the distribution of

shards' queue size yielded by both LBF and Metis. In contrast, the proposed CLPA demonstrates a more balanced distribution of transaction workloads.

VII. CONCLUSIONS

This paper aims to balance the transaction workloads across all shards in the account-based sharded blockchain. We first formulated the tradeoff between the cross-shard TX ratio and workload balance as a network partition problem. To address this issue, we carefully designed a sharding protocol *ShardCutter*, which consists of an account partition algorithm and a fine-tuned account migration mechanism. The experimental results obtained from an open-sourced testbed demonstrate that the proposed protocol outperforms other baselines on throughput, TX confirmation latency, queue size of TX pool, cross-shard TX ratio, and the standard deviation of shards' transaction workload balance.

We hope our study in this article can provide insights for the blockchain sharding community. For example, future versions of Ethereum may find our protocol helpful for estimating its upcoming sharding feature. In the future, we also plan to embed our protocol in a real-world high-performance blockchain sharding system.

REFERENCES

- [1] C. Li, H. Huang, Y. Zhao, X. Peng, R. Yang, Z. Zheng, and S. Guo, "Achieving scalability and load balance across blockchain shards for state sharding," in *Proc. of 41st International Symposium on Reliable Distributed Systems (SRDS'22)*. IEEE, 2022, pp. 284–294.
- [2] H. Huang, Y. Lin, and Z. Zheng, "Account migration across blockchain shards using fine-tuned lock mechanism," in *Proc. of IEEE 43rd International Conference on Computer Communications (INFOCOM'24)*, 2024, pp. 1–10.
- [3] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A Secure Sharding Protocol For Open Blockchains," in *Proc. of ACM SIGSAC Conference on Computer and Communications Security (CCS'16)*, 2016, pp. 17–30.
- [4] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "Omniledger: A secure, scale-out, decentralized ledger via sharding," in *Proc. of IEEE 39th Symposium on Security and Privacy (SP'18)*, 2018, pp. 583–598.
- [5] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proc. of ACM SIGSAC Conference on Computer and Communications Security (CCS'18)*, 2018, pp. 931–948.
- [6] J. Hong, S. Guo, E. Zhou, J. Zhang, W. Chen, J. Liang, J. Zhang, and A. Zomaya, "Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering," in *Proc. of IEEE 42nd International Conference on Computer Communications (INFOCOM'23)*, 2023.
- [7] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proc. of International Conference on Management of Data (SIGMOD'19)*, 2019, pp. 123–140.
- [8] L. N. Nguyen, T. D. Nguyen, T. N. Dinh, and M. T. Thai, "Optchain: optimal transactions placement for scalable blockchain sharding," in *Proc. of IEEE 39th International Conference on Distributed Computing Systems (ICDCS'19)*, 2019, pp. 525–535.
- [9] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *Proc. of 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI'19)*, 2019, pp. 95–112.
- [10] L. Jia, Y. Liu, K. Wang, and Y. Sun, "Estuary: A low cross-shard blockchain sharding protocol based on state splitting," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 3, pp. 405–420, 2024.
- [11] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo, "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. of IEEE 41st International Conference on Computer Communications (INFOCOM'22)*, 2022, pp. 1968–1977.

- [12] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, pp. 1–32, 2014.
- [13] H. Huang, G. Ye, Q. Yang, Q. Chen, Z. Yin, X. Luo, J. Lin, J. Zheng, T. Li, and Z. Zheng, "Blockemulator: An emulator enabling to test blockchain sharding protocols," *IEEE Transactions on Services Computing (TSC)*, vol. 18, no. 2, pp. 690–703, 2025.
- [14] S. Nakamoto, "Bitcoin: A peer-to-peer cash system," Tech. Rep., 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [15] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière, "Shard scheduler: object placement and migration in sharded account-based blockchains," in *Proc. of ACM Conference on Advances in Financial Technologies (AFT'21)*, 2021, pp. 43–56.
- [16] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.
- [17] Y. Zhang, S. Pan, and J. Yu, "Txallo: Dynamic transaction allocation in sharded blockchain systems," in *Proc. of IEEE 39th International Conference on Data Engineering (ICDE'23)*, 2023, pp. 721–733.
- [18] M. Li, W. Wang, and J. Zhang, "Lb-chain: Load-balanced and low-latency blockchain sharding via account migration," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 10, pp. 2797–2810, 2023.
- [19] S. Zhang and J.-H. Lee, "Double-spending with a sybil attack in the bitcoin decentralized network," *IEEE transactions on Industrial Informatics (TII)*, vol. 15, no. 10, pp. 5715–5722, 2019.
- [20] S. Gao, Z. Li, Z. Peng, and B. Xiao, "Power adjusting and bribery racing: Novel mining attacks in the bitcoin system," in *Proc. of the ACM SIGSAC Conference on Computer and Communications Security (CCS'19)*, 2019, pp. 833–850.
- [21] M. Li, Y. Lin, J. Zhang, and W. Wang, "Cochain: High concurrency blockchain sharding via consensus on consensus," in *Proc. of IEEE 42nd International Conference on Computer Communications (INFOCOM'23)*, 2023, pp. 1–10.
- [22] Z. Cai, J. Liang, W. Chen, Z. Hong, H.-N. Dai, J. Zhang, and Z. Zheng, "Benzene: Scaling blockchain with cooperation-based sharding," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 2, pp. 639–654, 2022.
- [23] H. Huang, X. Peng, Y. Lin, M. Xu, G. Ye, Z. Zheng, and S. Guo, "Scheduling most valuable committees for the sharded blockchain," *IEEE/ACM Transactions on Networking (ToN)*, vol. 31, no. 6, pp. 3284–3299, 2023.
- [24] Y. Tao, B. Li, J. Jiang, H. C. Ng, C. Wang, and B. Li, "On sharding open blockchains with smart contracts," in *Proc. of IEEE 36th International Conference on Data Engineering (ICDE'20)*, 2020, pp. 1357–1368.
- [25] E. Fynn, A. Bessani, and F. Pedone, "Smart contracts on the move," in *Proc. of 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'20)*, 2020, pp. 233–244.
- [26] M. Li, Y. Lin, J. Zhang, and W. Wang, "Jenga: Orchestrating smart contracts in sharding-based blockchain for efficient processing," in *Proc. of IEEE 42nd International Conference on Distributed Computing Systems (ICDCS'22)*, 2022, pp. 133–143.
- [27] Z. Hong, S. Guo, P. Li, and W. Chen, "Pyramid: A layered sharding blockchain system," in *Proc. of IEEE 40th International Conference on Computer Communications (INFOCOM'21)*, 2021, pp. 1–10.
- [28] X. Qi, "S-store: A scalable data store towards permissioned blockchain sharding," in *Proc. of IEEE 41st International Conference on Computer Communications (INFOCOM'22)*, 2022, pp. 1978–1987.
- [29] P. Zheng, Q. Xu, Z. Zheng, Z. Zhou, Y. Yan, and H. Zhang, "Meepo: Sharded consortium blockchain," in *Proc. of IEEE 37th International Conference on Data Engineering (ICDE'21)*, 2021, pp. 1847–1852.
- [30] Y. Xu, T. Slaats, and B. Döder, "Poster: Unanimous-majority-pushing blockchain sharding throughput to its limit," in *Proc. of ACM SIGSAC Conference on Computer and Communications Security (CCS'22)*, 2022, pp. 3495–3497.
- [31] M. J. Amiri, D. Agrawal, and A. El Abbadi, "Sharper: Sharding permissioned blockchains over network clusters," in *Proc. of International Conference on Management of Data (SIGMOD'21)*, 2021, pp. 76–88.
- [32] M. Castro, B. Liskov *et al.*, "Practical byzantine fault tolerance," in *Proc. of USENIX Symposium on Operating Systems Design and Implementation (OSDI'99)*, 1999, pp. 173–186.
- [33] W. Fan, R. Jin, M. Liu, P. Lu, X. Luo, R. Xu, Q. Yin, W. Yu, and J. Zhou, "Application driven graph partitioning," in *Proc. of International Conference on Management of Data (SIGMOD'20)*, 2020, pp. 1765–1779.
- [34] Karypis, George, Kumar, and Vipin, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.
- [35] J. Ugander and L. Backstrom, "Balanced label propagation for partitioning massive graphs," in *Proc. of the sixth ACM international conference on Web search and data mining (WSDM'13)*, 2013, pp. 507–516.
- [36] D. P. Williamson and D. B. Shmoys, *The design of approximation algorithms*. Cambridge university press, 2011.
- [37] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical review E*, vol. 76, no. 3, p. 036106, 2007.
- [38] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [39] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," *Proc. of the national academy of sciences*, vol. 105, no. 4, pp. 1118–1123, 2008.
- [40] H. Huang, Z. Yin, Q. Chen, G. Ye, X. Peng, Y. Lin, Z. Zheng, and S. Guo, "BrokerChain: A Blockchain Sharding Protocol by Exploiting Broker Accounts," *IEEE/ACM Transactions On Networking (ToN)*, pp. 1–16, 2025.
- [41] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *Proc. of 40th annual symposium on foundations of computer science (cat. No. 99CB37039)*. IEEE, 1999, pp. 120–130.



Huawei Huang (SM'22) is a professor at Sun Yat-Sen University, China. He received his Ph.D. in Computer Science and Engineering from the University of Aizu, Japan. His research interests mainly include blockchain systems and protocols. He has served as a guest editor for multiple special issues on blockchain at IEEE JSAC, OJ-CS, and IET Blockchain. He was also a TPC chair for several blockchain conferences and symposiums.



Jian Zheng is a Ph.D. student at Sun Yat-sen University, China. His research interest mainly includes blockchain.



Xuanye Zhu is an undergraduate student at Sun Yat-sen University, China. His research interest mainly includes blockchain. He is an intern in Prof. Huang's research team.



Yue Lin received his master's degree in the School of Computer Science and Engineering, Sun Yat-Sen University. His research interests mainly include Blockchain.



Ting Cai received the Ph.D. from Sun Yat-sen University, Guangzhou, China, in 2022. From 2009 to 2012, she was an Engineer with the Sinotrans & CSC Holdings Co. Ltd., Beijing, China. From 2012 to 2018, she was a Lecturer at Chongqing University of Posts and Telecom, Chongqing, China. She is an Associate Professor at the School of Computer Science, Hubei University of Technology, Wuhan, China. Her research interests include blockchain, deep reinforcement learning, Internet of Things security, and edge/cloud computing.

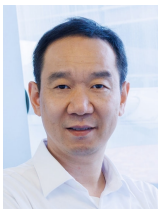


Lu Zhou received B.S. and M.S. degrees in computer science from Shandong University in 2012 and 2015, and the Ph.D. in computer science from the University of Aizu in 2019, respectively. She is a professor at Nanjing University of Aeronautics and Astronautics, China. Her current research interests include cryptographic engineering and blockchain applications. Her research is supported by the National Natural Science Foundation of China and the Natural Science Foundation of Jiangsu Province.



Zibin Zheng (SM'16-F'23) received a Ph.D. from the Chinese University of Hong Kong, Hong Kong, in 2012. He is a Professor at the School of Software Engineering, Sun Yat-Sen University, China. His current research interests include service computing, blockchain, and cloud computing. Prof. Zheng was a recipient of the Outstanding Ph.D. Dissertation Award of the Chinese University of Hong Kong in 2012, the ACM SIGSOFT Distinguished Paper Award at ICSE in 2010, the Best Student Paper Award at ICWS2010, and the IBM Ph.D. Fellowship

Award in 2010. He was a PC member for IEEE CLOUD, ICWS, SCC, ICSOC, and SOSE.



Song Guo (M'02-SM'11-F'20) is a full professor in the Department of Computer Science and Engineering (CSE) at the Hong Kong University of Science and Technology (HKUST). Before joining HKUST in 2023, he was a Professor at The Hong Kong Polytechnic University. He received his PhD degree in computer science from the University of Ottawa. He has authored or co-authored over 450 papers in major conferences and journals. His research interests include big data, cloud and edge computing, mobile computing, and distributed systems. Prof.

Guo received the 2019 TCBD Best Conference Paper Award, the 2018 IEEE TCGCC Best Magazine Paper Award, the 2017 IEEE SYSTEMS JOURNAL Annual Best Paper Award, and six other Best Paper Awards from IEEE/ACM conferences. He was an IEEE Communications Society Distinguished Lecturer. He has served as an Associate Editor of IEEE TPDS, IEEE TCC, IEEE TETC, etc. He also served as the general and program chair for numerous IEEE conferences. He currently serves on the Board of Governors of the IEEE Communications Society.